

TataStrive Analytics: Engagement Pipeline Masterclass

Introduction & Abstract

This manual is dedicated exclusively to the **Engagement Pipeline (Classroom Session Tracking)** of TataStrive Analytics. The Engagement logic is a specialized Computer Vision engine designed solely for monitoring **Behavior, Posture, and Participation**. It ignores the historical identities of the individuals, focusing entirely on answering the question: *Are the students actively participating in this specific session?*

By leveraging the **Ultralytics YOLOv8** and **BoT-SORT** frameworks running on local edge devices, the system tracks individuals temporally, calculates posture-based engagement weights, stitches fragmented tracks hierarchically, and streams structured class dynamic data to Google BigQuery.

Phase 0: Repository Map (Engagement Components)

Logical Component	Handlers & Core Files	Location / Path
Worker Orchestration	classroom_worker.py	app/workers/
Telemetry & Math	stitch_logic.py, group_by_session.py	classroom_analysis/
Vision Models	yolov8m.pt, yolov8n-pose.pt, osnet_x1_0_msmt17.pt	Root / Models/
Data Sync Engine	bigquery_sync.py	app/
UI Configuration	classroom_tab.py	app/ui/

Part 1: The Big Picture (System Architecture)

Step 1: Understand the Pipeline

The Engagement pipeline acts as an event-driven ingestion system triggered by the **Classroom Tab** folder listener.

- Detection & Pose:** YOLOv8m extracts body bounding boxes, while YOLOv8n-pose extracts skeletal keypoints to infer physical posture (e.g., writing, raising hand).
- Temporal Tracking:** BoT-SORT generates a visual embedding (via osnet) to assign a temporary tracking ID (e.g., Track_5) so the system remembers the student as they move around the room.
- Telemetry Calculation:** Engagement scores are calculated mathematically based on the inferred posture states.
- Aggregation:** Results are stitched hierarchically to resolve identity fragmentation,

generating a final JSON payload (`class_dynamics_report.json`).

5. **Cloud Synchronization:** The `BigQuerySyncService` hashes the JSON, deduplicates it, and streams it to Google Cloud.

Step 2: Visualize the System Flow Sequence

```
sequenceDiagram
    participant UI as Classroom UI (QTimer)
    participant Worker as ClassroomWorker
    participant YOLO as YOLOv8 (Bodies/Pose)
    participant SORT as BoT-SORT (Tracker)
    participant Math as Stitch & Session Aggregation
    participant Sync as BigQuerySyncService

    UI->>UI: Poll folder every 5 seconds
    UI->>Worker: Trigger on stable video
    Worker->>YOLO: Pass Frame (with FRAME_SKIP)
    YOLO->>YOLO: Extract Bounding Boxes & Keypoints
    YOLO->>SORT: Pass visual crops
    SORT-->>Math: Assign temporary Track_ID (ID_5)
    Worker->>Math: Calculate Activity & Attention Weights
    Math->>Math: Stitch broken tracks (Time/Space/Sim)
    Math->>Math: Group behaviors into Class Modes
    Worker-->>UI: Output class_dynamics_report.json
    UI->>Sync: Trigger Auto-Sync
    Sync->>Sync: SHA-256 Hash to prevent double-upload
```

Step 3: Trace the Payload Evolution

```
// 1. Raw YOLOv8 Extraction (Internal Memory)
{
  "track_id": 12,
  "pose_keypoints": {"nose": [100, 200], "wrist": [110, 250]},
  "inferred_activity": "writing"
}

// 2. Continuous Engagement Scoring (Internal Memory)
{
  "track_id": 12,
  "engagement_score": 0.85,
  "classification": "engaged"
}

// 3. Worker Output (Local JSON Dump)
{
  "hourly_probes": [
    {
      "time_slice": "09:00 - 09:15",
      "avg_engagement": 0.76,
      "act_writing_pct": 0.45,
      "class_mode": "Active Participation"
    }
  ]
}

// 4. BigQuery Sync Flattening (Sent to Cloud)
{
  "center_id": "CenterAlpha",
  "sync_timestamp": "2026-06-02T10:00:00.000Z",
  "time_slice": "09:00 - 09:15",
  "avg_engagement": 0.76,
```

```
"act_writing_pct": 0.45,  
"class_mode": "Active Participation"  
}
```

Step 4: Review Architectural Strategies

1. **Frame Skipping Strategy:** Edge execution of YOLOv8m and YOLOv8n-pose is incredibly taxing. By implementing a FRAME_SKIP strategy, the pipeline avoids running full inference on every single frame, extrapolating tracked positions in the intermediate frames to achieve near real-time processing on low-end CPUs.
 2. **Groq VLM Integration:** Extracting contextual metadata (like the Classroom Name or Timestamp) directly from the CCTV feed's burned-in OSD (On-Screen Display) allows the system to remain decoupled from complex API integrations with the actual NVR/DVR camera systems.
-

Part 2: The Toolbelt (Prerequisites & Infrastructure)

Step 1: Standalone Executable Packaging

[!NOTE]

This application uses PyInstaller, NOT Docker. Virtualization introduces overhead and hardware-passthrough complexity on Windows machines. The Engagement pipeline relies heavily on high-speed CPU instruction sets for PyTorch execution, meaning a standalone .exe built via PyInstaller is the optimal deployment strategy.

Step 2: Infrastructure Sandbox

1. **Google BigQuery:** Stores the massive influx of daily engagement rows.
2. **Streaming Inserts:** The app uses `insert_rows_json` rather than batch load jobs to ensure immediate reflection on dashboards.

Step 3: AI Frameworks

1. **Ultralytics (YOLOv8):** Drives the body detection and pose estimation networks.
 2. **BoT-SORT / Torchreid:** Provides the `osnet_x1_0_msmt17` model used to extract clothing/visual embeddings for temporal tracking.
 3. **Groq API (Llama3 Vision):** Serves as an affordable, ultra-low latency Vision Language Model for OSD metadata extraction.
-

Part 3: The Code Deep Dive (ClassroomWorker)

Step 1: Locate the Handlers

`app/workers/classroom_worker.py` acts as the execution brain, backed by `classroom_analysis/stitch_logic.py` for identity resolution.

Step 2: Code Walkthrough

1. Instantiated by `app.ui.classroom_tab.py`.
2. Initializes `FaceEngagementAnalyzerWithCallbacks`.
3. Loops over active frames, passing them through YOLOv8m (Bodies) and YOLOv8n-pose (Pose).
4. Passes image crops to BoT-SORT to assign sequential Tracking IDs.
5. `RuntimeAnchorManager` caches the ID to the seat coordinates to prevent tracking fragmentation when a student stands up or sits down.
6. Executes `calculate_engagement_score()`.
7. At video completion, `group_by_session.py` and `stitch_logic.py` synthesize the frame-level tracking data into time-sliced probes.

Step 3: Telemetry & Math (Scoring & Stitching)

The mathematical engine operates on exact configured weights and thresholds:

1. Engagement Score Weights | Feature Category | States & Weights | Formula | | --- | --- | --- | | **Activity** | `raising_hand` (1.0), `writing` (0.9), `listening` (0.6), `standing` (0.3), `walking` (0.1) | `Score += ActivityWeight` | | **Attention** | `focused` (0.3), `partially_focused` (0.15), `distracted` (0.0) | `Score += AttentionWeight` | | **Zone** | `front` (0.1), `middle_back` (0.05) | `Score += ZoneWeight` | | **Model Confidence** | Frame bounding box confidence | `Score += (Confidence * 0.1)` |

2. Hierarchical Stitching Thresholds | Threshold Variable | Default Value | Description | | --- | --- | --- | | `max_time_gap` | 600s | Tracks must occur within 10 minutes of each other to be stitched. | | `max_pixel_dist` | 200px | The bounding box must not have jumped massively across the room. | | `similarity_threshold` | 0.75 | The BoT-SORT Re-ID visual embedding must be highly correlated (> 0.75). |

Step 4: Configure Environment Variables

Variable	Required	Description
GROQ_API_KEY	Optional	Used by the VLM for automated metadata extraction from the video feed.
GOOGLE_APPLICATION_CREDENTIALS	Yes	Path to the BigQuery service account JSON for cloud syncing.

Step 5: "What-If" Scenario (Track Collision)

What if a student stands up, swaps seats, and breaks their visual tracking ID? The system utilizes a `RuntimeAnchorManager`. It "anchors" the student's ID to their original seat coordinates. Even if the visual Re-ID fails because they put on a jacket, the anchor manager forces the broken track to

map back to the original ID based on spatial proximity when they return to their seat, preventing massive ID fragmentation.

Part 4: Operations & Troubleshooting

Step 1: Build & Deployment Commands

```
python -m venv env
.\env\Scripts\activate
pip install -r requirements_app.txt
python build_exe.py
iscc installer\TataStriveAnalytics.iss
```

Step 2: Debugging Guide (Log Analysis)

- ImportError: DLL load failed:** Indicates the target machine is missing the Microsoft Visual C++ Redistributable, causing `onnxruntime` or `torch` to panic. Run the bundled `vc_redist.x64.exe`.
- Groq Authentication Error:** If the VLM metadata logs a 401 Unauthorized, the `GROQ_API_KEY` in `.env` is missing or expired. The worker will fall back gracefully to file creation timestamps.

Step 3: Common Failures Quick Reference

Symptom	Code / System Cause	Resolution
App Freezes on "Loading Models..."	PyTorch is initializing YOLOv8 weights into system memory.	Normal behavior on low-end CPUs. Do not force close; allow 10-15 seconds.
BigQuery Code 403	google-auth missing credentials.json.	Place JSON inside app/Creds/credentials.json.
Videos skipped by worker	Video file size is still changing.	The QTimer requires the file size to remain completely static across two 5-second polling intervals to ensure the CCTV recording is not corrupted.

Appendices

Appendix A: Event Field Reference (BigQuery)

Field Name	Type	Description
time_slice	STRING	The temporal bounds of the probe (e.g., 09:00 -

Field Name	Type	Description
		09:15).
act_listening_pct	FLOAT	Flatted representation of the proportion of students listening.
att_focused_pct	FLOAT	Flatted representation of the proportion of students focused.
overall_engagement_score	FLOAT	Final mathematical average of the Session.
class_mode	STRING	Macro-aggregation (e.g., Active Participation, Passive Focus).

Appendix B: Getting Started (For New Developers)

Because both the Attendance and Engagement pipelines run from the exact same application and .exe, the core environment setup is identical.

For the full setup guide (including virtual environments, API keys, and BigQuery credentials), please refer to the centralized [MASTERCLASS.md - Getting Started Guide](#).

Once your environment is configured and running (python app/main.py):

1. In the UI, navigate explicitly to the **Classroom Tab**.
2. Select your video input folder and output destination.
3. The folder listener will automatically trigger classroom_worker.py when stable videos arrive.